List of LP – III problem statements for Practice

A1: Write recursive and non-recursive programs to calculate Fibonacci numbers and analyze time and space complexity.

B1: Predict Uber ride fare: preprocess data, identify outliers, check correlation, train Linear Regression and Random Forest Regression, and compare R^2 and RMSE.

A1: Write recursive and non-recursive programs to calculate Fibonacci numbers and analyze time and space complexity.

B2: Classify email as spam or not spam using K-Nearest Neighbors and Support Vector Machine; analyze performance.

A1: Write recursive and non-recursive programs to calculate Fibonacci numbers and analyze time and space complexity.

B3: Build a neural-network classifier to predict bank customer churn within six months; split and normalize data, train the model, and print accuracy and confusion matrix.

A1: Write recursive and non-recursive programs to calculate Fibonacci numbers and analyze time and space complexity.

B4: Implement Gradient Descent to find a local minimum of $y = (x + 3)^2$ starting at $x = 2$.

A1: Write recursive and non-recursive programs to calculate Fibonacci numbers and analyze time and space complexity.

B5: Apply K-Means or hierarchical clustering to sales_data_sample.csv; determine the number of clusters using the elbow method.

A2: Implement Huffman Encoding using a greedy strategy.

B1: Predict Uber ride fare: preprocess data, identify outliers, check correlation, train Linear Regression and Random Forest Regression, and compare R^2 and RMSE.

A2: Implement Huffman Encoding using a greedy strategy.

B2: Classify email as spam or not spam using K-Nearest Neighbors and Support Vector Machine; analyze performance.

A2: Implement Huffman Encoding using a greedy strategy.

B3: Build a neural-network classifier to predict bank customer churn within six months; split and normalize data, train the model, and print accuracy and confusion matrix.

A2: Implement Huffman Encoding using a greedy strategy.

B4: Implement Gradient Descent to find a local minimum of $y = (x + 3)^2$ starting at $x = 2$.

A2: Implement Huffman Encoding using a greedy strategy.

B5: Apply K-Means or hierarchical clustering to sales_data_sample.csv; determine the number of clusters using the elbow method.

A3: Solve the 0-1 Knapsack problem using dynamic programming or branch and bound.

B1: Predict Uber ride fare: preprocess data, identify outliers, check correlation, train Linear Regression and Random Forest Regression, and compare R^2 and RMSE.

A3: Solve the 0-1 Knapsack problem using dynamic programming or branch and bound.

B2: Classify email as spam or not spam using K-Nearest Neighbors and Support Vector Machine; analyze performance.

A3: Solve the 0-1 Knapsack problem using dynamic programming or branch and bound.

B3: Build a neural-network classifier to predict bank customer churn within six months; split and normalize data, train the model, and print accuracy and confusion matrix.

A3: Solve the 0-1 Knapsack problem using dynamic programming or branch and bound.

B4: Implement Gradient Descent to find a local minimum of $y = (x + 3)^2$ starting at $x = 2$.

A3: Solve the 0-1 Knapsack problem using dynamic programming or branch and bound.

B5: Apply K-Means or hierarchical clustering to sales_data_sample.csv; determine the number of clusters using the elbow method.

A4: Design an n-Queens matrix with the first queen already placed; use backtracking to place the remaining queens.

B1: Predict Uber ride fare: preprocess data, identify outliers, check correlation, train Linear Regression and Random Forest Regression, and compare R^2 and RMSE.

A4: Design an n-Queens matrix with the first queen already placed; use backtracking to place the remaining queens.

B2: Classify email as spam or not spam using K-Nearest Neighbors and Support Vector Machine; analyze performance.

A4: Design an n-Queens matrix with the first queen already placed; use backtracking to place the remaining queens.

B3: Build a neural-network classifier to predict bank customer churn within six months; split and normalize data, train the model, and print accuracy and confusion matrix.

A4: Design an n-Queens matrix with the first queen already placed; use backtracking to place the remaining queens.

B4: Implement Gradient Descent to find a local minimum of $y = (x + 3)^2$ starting at $x = 2$.

A4: Design an n-Queens matrix with the first queen already placed; use backtracking to place the remaining queens.

B5: Apply K-Means or hierarchical clustering to sales_data_sample.csv; determine the number of clusters using the elbow method.

A5: Analyze Quick Sort using deterministic and randomized variants.

B1: Predict Uber ride fare: preprocess data, identify outliers, check correlation, train Linear Regression and Random Forest Regression, and compare R^2 and RMSE.

A5: Analyze Quick Sort using deterministic and randomized variants.

B2: Classify email as spam or not spam using K-Nearest Neighbors and Support Vector Machine; analyze performance.

A5: Analyze Quick Sort using deterministic and randomized variants.

B3: Build a neural-network classifier to predict bank customer churn within six months; split and normalize data, train the model, and print accuracy and confusion matrix.

A5: Analyze Quick Sort using deterministic and randomized variants.

B4: Implement Gradient Descent to find a local minimum of $y = (x + 3)^2$ starting at $x = 2$.

A5: Analyze Quick Sort using deterministic and randomized variants.

B5: Apply K-Means or hierarchical clustering to sales_data_sample.csv; determine the number of clusters using the elbow method.

A1: Write recursive and non-recursive programs to calculate Fibonacci numbers and analyze time and space complexity.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

A1: Write recursive and non-recursive programs to calculate Fibonacci numbers and analyze time and space complexity.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.

A2: Implement Huffman Encoding using a greedy strategy.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

A2: Implement Huffman Encoding using a greedy strategy.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.

A3: Solve the 0-1 Knapsack problem using dynamic programming or branch and bound.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

A3: Solve the 0-1 Knapsack problem using dynamic programming or branch and bound.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.

A4: Design an n-Queens matrix with the first queen already placed; use backtracking to place the remaining queens.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

A4: Design an n-Queens matrix with the first queen already placed; use backtracking to place the remaining queens.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.

A5: Analyze Quick Sort using deterministic and randomized variants.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

A5: Analyze Quick Sort using deterministic and randomized variants.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.

B1: Predict Uber ride fare: preprocess data, identify outliers, check correlation, train Linear Regression and Random Forest Regression, and compare R^2 and RMSE.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

B1: Predict Uber ride fare: preprocess data, identify outliers, check correlation, train Linear Regression and Random Forest Regression, and compare R^2 and RMSE.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.

B2: Classify email as spam or not spam using K-Nearest Neighbors and Support Vector Machine; analyze performance.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

B2: Classify email as spam or not spam using K-Nearest Neighbors and Support Vector Machine; analyze performance.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.

B3: Build a neural-network classifier to predict bank customer churn within six months; split and normalize data, train the model, and print accuracy and confusion matrix.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

B3: Build a neural-network classifier to predict bank customer churn within six months; split and normalize data, train the model, and print accuracy and confusion matrix.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.

B4: Implement Gradient Descent to find a local minimum of $y = (x + 3)^2$ starting at $x = 2$.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

B4: Implement Gradient Descent to find a local minimum of $y = (x + 3)^2$ starting at $x = 2$.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.

B5: Apply K-Means or hierarchical clustering to sales_data_sample.csv; determine the number of clusters using the elbow method.

C1: Write and deploy a bank account smart contract on a test network supporting deposit, withdrawal, and balance display.

B5: Apply K-Means or hierarchical clustering to sales_data_sample.csv; determine the number of clusters using the elbow method.

C2: Write and deploy a Solidity student-data smart contract using structures and arrays; observe transaction fee and gas values.